

The Core Stack & Environment

- **MERN:** A "full-stack" JavaScript framework consisting of **M**ongoDB (database), **E**xpress.js (backend framework), **R**eact (frontend library), and **N**ode.js (runtime environment).
- **Node.js:** A runtime environment that lets you run JavaScript on your computer or a server, rather than just inside a web browser.
- **Api :** An API is a set of rules and protocols that allows one software application to talk to another.
- **In Web Dev:** When your React frontend needs data from your MongoDB database, it sends a request to an **API endpoint** (like `GET /api/users`) on your Express server.
- **Express:** A minimal web application framework for Node.js. It handles things like "routes" (what happens when you go to `/api/users`) and server logic.
- **NPM (Node Package Manager):** A giant library of pre-written code (packages) that you can download and use in your projects.
- **NPX:** A tool used to *execute* packages without permanently installing them. It's most commonly used to run generators like `npx create-react-app`.
- **Package:** A reusable piece of code or a library (like a folder of tools) that you download via NPM to add specific functionality to your app.

The React Ecosystem

- **React:** A JavaScript library for building user interfaces based on UI components.
- **Component:** The building block of React. Think of it as a custom, reusable HTML element (e.g., a or).
- **JSX (JavaScript XML):** A syntax extension that allows you to write HTML-like code directly inside your JavaScript.
- **DOM (Document Object Model):** The browser's internal "tree" representation of your webpage. Updating it is traditionally slow.
- **VDOM (Virtual DOM):** A lightweight copy of the real DOM. React updates the VDOM first, compares it to the real DOM, and only changes the parts that actually moved or changed. This makes React very fast.
- **State:** A JavaScript object that holds data specific to a component. When the "state" changes, the component re-renders to show the new data.
- **State Management:** The practice of managing data across many components. While small apps use local state, larger apps use tools like Redux or Context API to keep data synchronized.
- **ES6 Modules:** The modern way to share code between files using import and export statements.

Execution & Logic

- **Synchronous:** Code that runs line-by-line in order. Each line must finish before the next one starts.
- **Asynchronous (Async):** Code that starts a task (like fetching data from a database) and moves on to the next line without waiting for it to finish.
- **Await:** A keyword used inside async functions to tell JavaScript: "Pause here until this specific task is finished, then continue."
- **Filter:** A JavaScript array method that creates a new array containing only the items that pass a specific "test."
- **Map:** A JavaScript array method that loops through an array and transforms every item into something else (e.g., turning a list of names into a list of HTML tags).

Tooling & Bundlers

- **Webpack:** A "module bundler." It takes all your different files (JS, CSS, Images) and packages them into one or two tiny files that the browser can understand quickly.
- **CRA (Create React App):** A deprecated (but famous) tool that set up a React project with all the configurations (like Webpack and Babel) hidden behind the scenes.
- **Consists of:** Webpack, Babel (translates new JS to old JS), Autoprefixer, and a development server.
- **Vite:** The modern, much faster successor to CRA. It uses a different way of handling files during development, making the app start almost instantly.

1. What is MVC Architecture?

MVC stands for:

- 👉 **M – Model**
- 👉 **V – View**
- 👉 **C – Controller**

It is a way to **organize code** in an application.

2. Model (M)

What is Model?

Model handles data and business logic.

Simple words:

- Stores data
- Talks to database
- Applies rules

Example:

- User data
- Product list
- API response

 **Example:**

```
User = { name: "Ram", age: 25 }
```



3. View (V)

What is View?

View is what the user sees on the screen.

Simple words:

- UI
- HTML / CSS
- Buttons, text, forms



Example:

- Login page
 - Dashboard
 - Profile page
-



4. Controller (C)

What is Controller?

Controller connects Model and View.

Simple words:

- Takes user actions
- Updates data
- Decides what to show



Example:

- Button click handler
 - API call logic
-

MVC Flow (Easy)

1. User interacts with **View**
 2. **Controller** handles action
 3. **Model** updates data
 4. **View** updates UI
-

5. Role of React in MVC Architecture

? Where does React fit?

👉 React is the **VIEW (V)** part of MVC

Why React is View?

- React builds **UI**
- React shows data on screen
- React updates UI when data changes

📌 React does **NOT**:

- Manage database
 - Handle backend logic
-

Simple mapping

MVC Part	Role	Example
Model	Data	API / DB
View	UI	React
Controller	Logic	Node / Spring / Django

React + MVC Example

- **Model** → Backend API (JSON data)
- **Controller** → Backend logic / API calls
- **View** → React Components